

Open-Source AI-Native Analog IC Design Platform

A Virtuoso / Spectre Alternative — Complete Specification, Zero to Everything

What this document is

This is a single, start-to-finish walkthrough of the platform: what it is, why it's being built this way, everything it will do from an empty repository to a complete Sky130 design flow, and the governance model that keeps an AI-heavy build honest along the way. It synthesizes and supersedes nothing — it's a readable companion to two working documents: the **Master Build Plan (v3.1 + addendum)**, which specifies the architecture and stage-by-stage plan, and the **AI Prompt Playbook (v1.1 + addendum)**, which specifies exactly how an AI coding agent is directed, checked, and gated at each stage.

Read this first if: you're a collaborator, reviewer, or future version of the builder who needs the whole shape of the thing in one sitting, without switching between two source documents.

Source documents	Master Build Plan v3.1 (+ addendum) · AI Prompt Playbook v1.1 (+ addendum)
Target user (v1)	Open-source-silicon designers, researchers, and students working in Sky130
Status	Architecture frozen and buildable. Stage 0 is the next concrete action.
Scope of this document	Product vision, architecture, full data model, stage roadmap, AI governance, tooling, and version ladder — everything, end to end

Contents

What this document is 1

Contents 2

1. Executive Summary 3

2. Product Vision & Positioning 3

How this compares to what already exists 4

3. Architecture Overview 5

The loop 5

The non-negotiable AI boundary 5

One engine, five clients 5

4. Foundational Technical Decisions 6

LLM provider abstraction 6

Reproducibility, in full 7

5. Core Data Model 8

6. The Design Engine API 9

API versioning 9

7. Stage-by-Stage Build Roadmap 10

8. The Design Knowledge Base — the Actual Moat 11

9. Benchmark Ladder 11

10. AI Integration & Human-in-the-Loop Governance 12

The AI proposal workflow 12

What an AI proposal actually is 12

The grounding rule for AI explanations (Stage 5) 12

Mandatory human checkpoints 13

11. Version Ladder 14

12. Tooling & AI-Assisted Development Workflow 15

Core stack 15

Directing an AI coding agent through this build 15

Routing policy 16

Anti-patterns that will quietly kill an AI-heavy build 16

13. Where Things Stand, and What's Next

17

1. Executive Summary

This platform is an **open-source, AI-native analog IC design environment** for the Sky130 process — built headlessly first, with every graphical or AI surface treated as a client of one underlying engine, rather than as the product itself. The end goal is a genuinely usable open alternative to Virtuoso/Spectre for open-silicon work; the v1 goal is narrower and honest: a coherent, reproducible spec-to-signoff flow for a defined set of Sky130 circuits, not a professional-tool replacement on day one.

The central design decision is a loop — **spec** → **topology** → **sizing** → **simulation** → **measurement** → **optimization** → **verification** — implemented once, headlessly, and then exposed identically to a CLI, a Python API, a schematic GUI, and an AI copilot. None of those four gets a private shortcut into the simulator, the layout engine, or the underlying schema. That single rule — everything is a client of the Design Engine — is the most consequential architectural decision in the whole plan, more consequential than any framework choice.

The AI strategy follows directly from that: the LLM **proposes**, deterministic software **validates**, the simulator **evaluates**, a constraint engine **judgets**, and a human **approves**. The model is never trusted to act as an analog simulator, and it never writes SPICE or connectivity directly — it selects from known topology templates and proposes parameters, which is a meaningfully safer and more tractable problem than free-form circuit synthesis.

The moat, in one sentence

A reproducible, validated design engine with a growing, structured knowledge base of real experiments — successes and failures alike — that AI methods plug into, rather than a fresh one-shot generation every time with no memory of what already failed.

As of this document, the architecture has been through four review passes and is considered frozen and buildable. The next artifact that will teach the team more than another review is a failing test against real libngspice output on a hand-built inverter — not a fifth round of planning.

2. Product Vision & Positioning

Two real research systems as of 2026 are worth naming specifically: **AnalogCoder-Pro** (topology generation plus Bayesian-optimization sizing, with a multimodal diagnosis-and-repair loop, benchmarked on 13 circuit types) and **AaLLM** (an open-source multi-agent topology-and-sizing workflow with a RAG-based knowledge base). Both are legitimate, and both operate on one design at a time, with no persistent, growing, validated substrate underneath them.

The opportunity this platform is built around isn't "we also do AI topology and sizing" — that's now a populated research area. It's a reproducible, validated design engine with a growing knowledge base that AI methods, including ones like these, could plug into — rather than a fresh one-shot generation every time with no memory of what already failed.

How this compares to what already exists

Approach	Strength	Weakness
Xschem + ngspice	Open, mature	Fragmented UX, no shared data model
KLayout / Magic / Netgen	Strong open physical flow	Not one integrated environment
CACE	Real characterization automation	Not a complete design environment
AnalogCoder-Pro / AaLLM	Real end-to-end topology+sizing research	Per-design, no persistent validated substrate
Commercial EDA	Extremely mature, AI already added	Expensive, proprietary, AI retrofitted onto old workflows
This platform	Unified loop + growing knowledge base + AI constrained by validation	Initially limited PDK / circuit coverage

3. Architecture Overview

The build order is headless-first: the full design loop is implemented and tested with no graphical interface at all, and the GUI (Stage 7) is built last, as one more client. This forces the underlying engine to be genuinely complete and correct before any surface is layered on top of it.

The loop

spec → **topology** → **sizing** → **simulation** → **measurement** → **optimization** → **verification**

The non-negotiable AI boundary

What this architecture explicitly forbids:

AI → SPICE directly

✗ forbidden

What it requires instead:

AI → Design Engine → validation → simulator

✓ required

This is what makes every later “X is just a client of the engine” claim true instead of aspirational — the Design Engine API (Section 6) is the literal, written-down contract that CLI, Python API, GUI, and AI copilot all share, with no side doors into ngspice, KLayout, or the schema for any of them.

One engine, five clients

- **CLI** — scriptable access for batch runs and CI
- **Python API** — the programmatic surface everything else is built on
- **Schematic GUI** (Stage 7) — visual design entry, built last, held to a strict equivalence test against the API
- **AI copilot** (Stage 5–6) — explains failures, proposes topologies and sizings, never commits directly
- **Optimization engine & automated testbenches** — every trial is a first-class, logged Job

4. Foundational Technical Decisions

PDK

Sky130 first, behind a **PDKAdapter** interface — a second PDK later (GF180/IHP, V2.0) is the real test of whether that abstraction was genuine or a Sky130-shaped hole.

Simulator

ngspice via **libngspice** (the shared library, not shelling out to the ngspice binary), behind a **Simulator** interface.

Layout

KLayout as the primary backend, behind a **LayoutBackend** interface, with Magic and Netgen wired in as adapters behind the same interface.

Execution model

A **Job** record (id, type, status, inputs, artifacts) plus a simple local async task runner — one process, a small worker pool, a status table in SQLite. Explicitly not Celery, not Redis, not a distributed queue. The shape is right from day one so nothing built on top of it later needs a rewrite, but the implementation stays single-machine until real multi-user demand exists.

Units

Every value is stored internally in SI base units — farads, ohms, hertz, volts, amps, seconds. Human-readable formatting (“10 MHz”) is a display-layer concern only, never the stored representation. Cheap to build correctly now; expensive to retrofit.

Reproducibility

Docker/Podman with every dependency pinned (Sky130, ngspice, KLayout, Python), reduced to make `setup` && `make test` && `make run-example`.

AI architecture

The LLM is one client of the Design Engine. It proposes; it never commits directly.

LLM provider abstraction

The “LLM is a replaceable client” principle above is backed by an actual interface, not just an intention. Any call into a model — the Stage 5 explainer, the Stage 6 topology proposer, the embeddings model once Stage 6’s retrieval exists — goes through one abstraction:

```
LLMProvider (abstract)
|-- GeminiProvider
|-- OpenAIProvider
|-- LocalProvider      # self-hosted 7-8B model, 4-bit quant
`-- MockProvider       # deterministic, tests only -- never a live API

methods: generate(), structured_generate(), embed()

call metadata: model, provider, prompt_version, temperature,
               seed, token usage, timestamp, request id
```

This is what keeps “swap Gemini for a local model” a config change instead of a rewrite, and it’s the source of the provenance fields recorded on every AI action (Section 5).

Reproducibility, in full

A netlist hash plus a simulation-config hash plus a declared numerical tolerance is the correct *identity and comparison* framework — it decides whether two runs are “the same design,” without wrongly demanding byte-identical floating-point output. But identity isn’t the same thing as full reproducibility: two runs can share both hashes while running against a different simulator build or PDK revision underneath. The complete environment is captured alongside the hash:

- netlist hash
- simulation-config hash
- simulator version
- PDK version
- model-file hash
- container image digest
- random seed
- analysis settings
- declared numerical tolerance

5. Core Data Model

Everything the platform tracks is one connected schema — no subsystem keeps its own private record of a design, a run, or a decision. The validation gate applies uniformly: nothing, human-entered or AI-proposed, reaches ngspice without passing schema, connectivity, unit, and model-binding checks first.

Entity	What it captures
Project / Library / Cell / Schematic / Symbol / Instance / Port / Net / Parameter	The design itself — the structural backbone every other table hangs off of.
Technology, ModelBinding	Ties a design to a specific PDK and device models.
Specification	Hard constraints (must pass), soft objectives (better-is-better), and weighted objectives (e.g. a figure of merit combining gain × bandwidth / power) — each with its own tolerance and priority, evaluated across nominal → PVT corners → Monte Carlo, never nominal-only.
Testbench, Analysis, Job	What gets simulated, how, and the execution record for each run.
SimulationRun	Netlist, versions, inputs, raw output, and the full reproducibility record described in Section 4.
Measurement	Structured metrics with pass/fail against a Specification.
Experiment / OptimizationRun	One row per optimizer trial — success and failure alike.
Constraint, DesignRevision	Constraints on a design; revision history (branching/diff UI arrives with the GUI, Stage 7).
ErrorRecord	Every failure classified against a fixed taxonomy: Syntax → Schema → Netlist → SPICE convergence → Operating point → Constraint → PVT → Monte Carlo → DRC → LVS → PEX → Post-layout performance. This is what lets the AI explainer say “this is a bias-point headroom problem” instead of “simulation failed.”
Artifact	A pointer to an on-disk file — a waveform PNG, a layout screenshot, a report.
AIAction	Provenance for every AI-generated action: id, action_type, model_provider, model_name, model_version, prompt_version, input_context_hash, output_hash, source_artifacts, resulting_design_revision, human_decision, timestamp. Answers “why did this design exist,” “which model produced the bad topology,” and “did results improve after prompt v17.”

6. The Design Engine API

Implicit in earlier drafts (“the LLM is one client,” “the GUI is one more client”) but written down here as an actual contract — the surface every client calls, and the only path into ngspice, KLayout, or the schema:

```
create_project()  create_cell()  instantiate()
validate()        netlist()    simulate()
check_constraints() optimize()  run_drc()
extract()         compare()
```

CLI, Python API, GUI, and the AI copilot are all callers of this exact surface. This is a bigger architectural decision than any front-end framework choice, and it's the one to get right first.

API versioning

Because every client depends on this exact surface, it's treated as a versioned protocol from Stage 0 onward — **DesignEngine v0.1**. Additive changes (new optional parameters, new methods) don't bump the major version; anything that changes an existing method's signature or behavior does, and ships with a migration note in the same commit. `engine_api_version` is recorded alongside `SimulationRun` and `AIAction` rows, so a design revision's provenance includes which contract produced it — this matters most once the AI copilot and the GUI are both calling the same engine concurrently.

7. Stage-by-Stage Build Roadmap

The full path from an empty repository to a working, AI-assisted, physically-verified design flow, in eleven stages. ● marks a blocking human checkpoint; ● marks an advisory one — both are defined in full in Section 10.

Stage	Name	What gets built	Checkpoint
0	Architecture & Packaging	Docker image (pinned versions), Makefile (setup/test/run-example), interface stubs, Design Engine API skeleton, CI smoke test.	—
1	Circuit Kernel	Schema, netlist compiler, pre-sim validator, unit system live from line one.	● Golden netlist
2	Simulation Kernel	Simulator interface, NgspiceBackend, the Job runner, tolerance-based reproducibility.	● First waveform
3	Measurement & Specification Engine	Gain, bandwidth, phase margin, slew rate, power, offset, settling time — plus hard/soft/weighted Specification evaluation.	● Per metric
4	Optimization Layer	Optimizer interface, Optuna first, every trial logged to the Experiment Ledger. First demoable product — no GUI needed.	● Degenerate optima
4.5	Robustness	Extends Stage 3/4 across PVT corners and Monte Carlo, using the PDK's own statistical model cards.	● Any % claim
5	AI Diagnostics & Copilot	Deterministic failure classifier + LLM-written, citation-grounded explanations; optimizer-result narration.	● Ungrounded prose
6	Topology Intelligence	Topology templates linked to their historical experiments; AI retrieves and fills a template, never invents connectivity.	● Every proposal
7	Schematic UI	One more client of the Design Engine API; UI-vs-API netlist-hash and tolerance-based equivalence test.	—
8	Physical Design	Viewer → geometry → import → PCells → placement → DRC → LVS (Netgen) → extraction.	● First clean DRC/LVS
9	Signoff Loop	Post-layout parasitics feed back into Stage 3's measurement engine; pre/post-layout deltas reported side by side.	—
10	Expansion (V2.0+)	Multi-PDK, AI-assisted layout, surrogate models, RF/EM, cloud collaboration.	Sandbox gate

Reading the shape of it: Stages 0–3 build a correct, headless circuit-and-simulation kernel with no AI involved at all. Stages 4–4.5 make it capable of sizing a real circuit against a real spec, robustly. Stages 5–6 add AI — first as an explainer with no write access, then as a gated proposer that still can't commit anything itself. Stage 7 is the first GUI, held to the same correctness bar as the API. Stages 8–9 close the loop into physical layout and signoff. Stage 10 is explicitly post-V1.0 scope.

8. The Design Knowledge Base — the Actual Moat

Every simulation and optimization trial — success or failure — logs to the Experiment Ledger from Stage 2 onward: parameters, measurements, verdict, reproducibility hash. Once Stage 6 exists, topology templates link to their own historical experiments — which sizings worked, which didn't, for which spec.

The LLM, the optimizer, even ngspice itself are all replaceable. A growing, structured, reproducible library of Sky130 design experience tied to one validated schema is not something a competitor bolts on after the fact — and it's specifically what point-solution research tools (AnalogCoder-Pro, AaLLM, TopoSizing) don't have, because they're evaluated per-paper on fixed benchmarks, not as a running system that accumulates its own history.

The compounding asset
validated circuit engine + structured experiments + topology templates + accumulated design history + AI — not “LLM generates SPICE.”

9. Benchmark Ladder

Level	Circuit	Evaluated across
0	CMOS inverter	Nominal
1	Current mirror, differential pair	Nominal → PVT
2	Common-source amp, cascode amp	Nominal → PVT → Monte Carlo
3	Two-stage Miller op-amp	Nominal → PVT → Monte Carlo → Layout + PEX
4	OTA / comparator / LDO	V2.0
5	Bandgap / VCO	Stretch goal, later

10. AI Integration & Human-in-the-Loop Governance

The core risk in an AI-heavy build isn't code that crashes — crashes are self-announcing. It's code that runs, produces a plausible-looking number, and is quietly wrong: a netlist with a swapped node order that still simulates, a measurement formula off by a sign convention, a PCell that's geometrically non-physical but LVS-clean. None of these throw an exception. This section is the mechanism that catches them anyway.

The AI proposal workflow

```
AI proposes → schema validation → simulate → check_constraints  
→ human approval (accept / reject / compare) → commit
```

There is no code path from “all checks passed” to “committed” that skips the human decision — not a config flag, not an `--auto-approve` setting, not a “trusted mode.” AI never silently overwrites the current design.

What an AI proposal actually is

Not prose the system parses to guess intent — one specific machine-readable object, validated as data:

```
{  
  "topology_id": "two_stage_miller_v1",  
  "parameters": {"w1": 2.0e-6, "l1": 5.0e-7, "w2": 8.0e-6},  
  "reasoning": "...",  
  "evidence_ids": ["EXP-102", "EXP-117"],  
  "requested_spec_id": "SPEC-42"  
}
```

`topology_id` must resolve to a known template — there is no free-form connectivity entry point for the AI, in Stage 6 or ever in v1. Novel topology synthesis is explicitly deferred to the V2.0/V3.0 horizon (Section 11), not missing from v1 by accident. Every proposal is logged as an `AIAction` row before it reaches the human-approval gate — rejected and failed proposals are provenance data too, on the same principle as failed trials in the Experiment Ledger.

The grounding rule for AI explanations (Stage 5)

Every factual claim the explainer produces must be traceable to a specific `ErrorRecord` or `Measurement ID`. This is enforced as an actual check, not a prompt instruction: the explainer returns (`prose`, `cited_ids`); if `cited_ids` is empty, the system returns “Insufficient data to explain this failure” — it does not return prose. A plausible, uncited story is the exact failure mode this stage is architected against. The granularity is per claim, not per sentence — two sentences grounded by the same measurement share one citation.

Mandatory human checkpoints

Stage / Component	Trigger	What's verified	Type
Stage 1 — Netlist compiler	Golden reference netlist created	It's electrically correct by hand, before it's trusted as ground truth for every future comparison	Blocking
Stage 2 — First simulation	First transient waveform from libngspice	Waveform shape and values are sane, before this run becomes the template other testbenches copy	Blocking
Stage 3 — Measurement formulas	First implementation of each metric	Matches a hand calculation, once per metric, before it's trusted for automated runs	Blocking
Stage 4 — Optimizer results	Trial sits at a search-space boundary, or converges suspiciously fast	The spec wasn't mis-scoped in a way that let the optimizer find a degenerate solution	Advisory
Stage 4.5 — Robustness claims	Any "X% of Monte Carlo samples pass" statement	Sample count and corner distribution — small N can produce a misleadingly clean percentage	Blocking
Stage 5 — AI explanations	An explainer narrates a failure or optimizer result	The explanation cites a real ErrorRecord/Measurement, or says it can't	Advisory
Stage 6 — AI design proposals	Any AI-proposed topology or sizing, every time	Accept / reject / compare — never auto-committed even when all checks pass	Blocking
Stage 8 — DRC/LVS/PCells	First clean DRC+LVS pass on a benchmark circuit	Visual review in the KLayout viewer before the PCell generator is reused elsewhere	Blocking
Cross-cutting — PDK/model syntax	Unvalidated Sky130 model-binding or device syntax	Confirmed against PDK reference docs, never written from memory	Blocking

Operating rule: default to flagging over guessing. The first time the agent is about to trust output in one of these categories, it stops and raises a checkpoint instead of proceeding. Once a specific pattern is human-verified once, identical subsequent runs of that pattern don't need to re-flag — only new patterns or anomalous results from old ones. Flagging everything forever is as useless as flagging nothing.

11. Version Ladder

Version	Milestone	Scope
V0	Engineering proof	Stages 0–2, inverter only, Docker + regression test.
V0.1	Circuit engine	Stages 1–3 fleshed out, all Level 0–1 benchmarks, Design Engine API complete for headless use, no GUI.
V0.2	Optimization + robustness	Stage 4 + 4.5, Level 2 benchmarks, Experiment Ledger live.
V0.3	AI copilot	Stage 5.
V0.4	Schematic editor + topology intelligence	Stage 6 + Stage 7, built together — the human-approval workflow in Stage 6 needs a visual surface to approve against.
V0.5	Physical design + signoff	Stage 8 + 9.
V1.0	First real, coherent product	Everything above, integrated end to end: Sky130 → spec → topology → sizing → simulation → measurement → optimization → schematic → layout → DRC/LVS → PEX → post-layout verification, AI-assisted throughout. Not autonomous AI layout, not multi-PDK — explicitly out of scope here.
V2.0	Expansion	GF180/IHP via PDKAdapter, Level 4 benchmarks, surrogate models, larger Knowledge Base, AI-assisted layout, sandboxed execution.
V3.0	Autonomy	Autonomous topology exploration, advanced placement/routing, RF, collaboration.

12. Tooling & AI-Assisted Development Workflow

Core stack

Layer	Choice
Simulation	ngspice / libngspice (Xyce later) behind Simulator
Layout	KLayout (+ Magic/Netgen) behind LayoutBackend
Characterization	CACE, adopted directly
PDKs	Sky130 first, behind PDKAdapter
Optimization	Optuna first
Orchestration	Plain functions through Stage 5; LangGraph enters at Stage 6, and only there
LLM	Ollama/vLLM or a hosted API — always optional, via LLMPProvider
Testing	pytest + golden-netlist regression + CI from Stage 0

Directing an AI coding agent through this build

The build itself is meant to be largely AI-agent-driven, under a strict three-layer prompt structure: global rules (an `AGENTS.md` at the repo root — identity, the four non-negotiable laws, units, scope control, testing, error handling, honesty rules, and the checkpoint protocol), a per-stage brief, and a single commit-sized task prompt. One prompt equals one commit equals one testable claim; anything that would produce more than roughly 400 lines of new code gets split.

The four non-negotiable laws (`AGENTS.md`):

- Never mark a stage “done” on the agent's own judgment — done-when criteria are verified by a human.
- Never write Sky130 model-binding or device syntax from memory — read the PDK reference or ask.
- Never let anything reach ngspice without passing schema + connectivity + unit + model-binding validation.
- Never give a subsystem a private path to ngspice, KLayout, or the schema — everything routes through the Design Engine API. No side doors, ever.

A fifth, added rule governs AI provenance specifically: every model call goes through LLMPProvider with full call metadata; every AI-generated action logs an `AIAction` row before anything downstream trusts it; and an AI proposal crosses into the Design Engine as the structured object from Section 10, never as parsed prose.

Routing policy

Situation	Route to
New subsystem, no precedent in repo	Full-context build agent, strongest available model
Mechanical change following an existing pattern	Fast/cheap tier of the same agent
Anything touching PDK syntax or units	Full context, PDK docs attached, always
Explaining an error already understood	Don't ask the agent — read the traceback
Rewriting a stage “more cleanly”	Don't — it passes tests or it doesn't
Second opinion / adversarial review	A different model family than whoever wrote the code

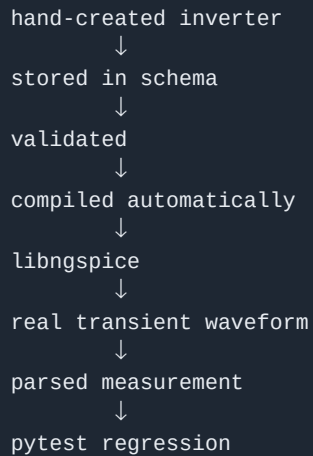
Anti-patterns that will quietly kill an AI-heavy build

- **The plausible number** — a reported result nobody actually simulated. Nothing crashed. Three stages get built on top of it.
- **Golden-reference drift** — the agent edits the golden file to make a test pass. Make golden files read-only in CI.
- **Silent unit coercion** — a float is a float; without typed quantities, the unit system is just a naming convention.
- **The helpful side door** — “I added a direct DB read in the UI for performance.” This is how the Design Engine API boundary dies. Reject on sight.
- **Checkpoint fatigue** — rubber-stamping checkpoints costs time and buys nothing. Either actually check, or remove the checkpoint.
- **Review-loop addiction** — planning has a declared endpoint. The next artifact that teaches something is a failing test from real output, not a fifth architecture review.
- **Multi-stage prompts** — asking one prompt to build two stages at once produces code that runs and is wrong in ways that only surface much later.

13. Where Things Stand, and What's Next

The architecture has been through four consecutive review passes, each of which caught something real. That streak doesn't have a natural stopping point in the abstract — a fifth review will always find another defensible addition, because “more rigor” has no ceiling on its own. The ceiling has to be declared. It has been: the plan is done. It's buildable as written, with the five addendum items woven in above.

The concrete next step, in order:



```
graph TD; A[hand-created inverter] --> B[stored in schema]; B --> C[validated]; C --> D[compiled automatically]; D --> E[libngspice]; E --> F[real transient waveform]; F --> G[parsed measurement]; G --> H[pytest regression];
```

Once that path works end to end on a real machine, it teaches more about whether the architecture is right than another twenty pages of planning would. Stage 0 and Stage 1 come before asking anything — human or AI — to review this document again.

One rule above all the rest: the agent proposes, the simulator decides, and you commit. Every governance mechanism in this document exists to enforce that ordering.

14. AI Feature Expansion Addendum

September 2026 - additive roadmap, no removal or rewrite of the original 17 pages. This appendix expands the AI feature set while preserving the project's core law: AI proposes, deterministic software validates, real EDA tools decide, and the human promotes trusted changes.

The existing specification already makes the LLM a client of the Design Engine, requires citation-grounded explanations, and gates AI design proposals through validation, simulation, constraints, and human approval. The expansion below keeps those invariants and adds a product roadmap around them.

Three corrections make the proposed roadmap materially safer and more technically useful:

- **Surrogates accelerate search; they never become signoff.** Out-of-domain or uncertain predictions fall back to SPICE, and every promoted design is simulated for real.
- **PDK porting is intent-aware, not minimum-length scaling.** Translate operating-point intent, legal voltage domains, current density, headroom, gm/Id-like behavior, matching, and capacitance - then re-optimize in the target PDK.
- **Multimodal layout debugging is report-first.** DRC rule IDs, marker geometry, LVS mismatch graphs, layer/net metadata, and extracted geometry are primary evidence; screenshots are supporting context.

Commercial focus

No AI roadmap can make a startup foolproof or guarantee financial success. The commercially defensible path is to make engineering measurably faster and more trustworthy while compounding a proprietary-by-use asset: validated experiment history, failures included, tied to reproducible designs.

Compounding loop

engine correctness -> validated experiments -> better retrieval/diagnostics/sensitivity -> smarter proposals and experiment planning -> fewer wasted runs -> more validated experiments -> stronger surrogate/porting/layout intelligence -> stronger product moat

14.1 AI roadmap mapped onto the existing build stages

ID	Capability	Earliest integration	Release	Hard trust boundary
AI-01	Grounded Error Explainer 2.0	Stage 5	V0.3	Every claim cites ErrorRecord/Measurement/Artifact or refuses
AI-02	Optimizer + trade-off narration	Stage 5	V0.3	Math and Pareto front computed deterministically
AI-03	Template topology proposer + KB RAG	Stage 6	V0.4	Known topology only; validate, simulate, human approve
AI-04	Conversational Spec-to-Constraint	After Stage 3; surface in Stage 5	V0.3/0.4	Creates SpecDraft; no silent defaults; human activates
AI-05	Multi-objective Pareto analyst	Stage 4/5	V0.3 enhancement	Saved deterministic ParetoSnapshot is truth
AI-06	Surrogate sizing accelerator	Post-V1	V2.0	Prediction never satisfies spec; SPICE verifies promotion
AI-07	PDK porting assistant	After second PDK adapter	V2.0	Target PDK validation, optimization, PVT/MC required
AI-08	Multimodal DRC/LVS debugger	After Stage 8	V2.0	Structured rule/mismatch evidence outranks image
AI-09	AI placement clustering	After layout constraints exist	V2.0	AI proposes constraints; layout engine draws geometry
AI-10	Autonomous topology exploration	Post-V2	V3.0	Restricted candidate IR + sandbox + human promotion

Scope discipline

AI-04 and AI-05 are lightweight additions around infrastructure that already exists for V1 and should not delay Stages 0-4. AI-06 through AI-10 remain post-V1 unless the core engine has already cleared its benchmark ladder. This avoids turning the roadmap into AI theater before the substrate is trustworthy.

Measured, not assumed

Autonomous topology exploration must not hard-code an assumed "99% fail / 1% pass" success rate. Record actual compile, simulation, constraint, and promotion rates by model/prompt/version.

14.2 V0.3-V0.4 intelligence: make the core workflow dramatically better

AI-01 - Grounded Error Explainer 2.0

Return structured supported findings, unknowns, evidence IDs, confidence, and proposed diagnostic tests. A proposed test is explicitly a next experiment - not a factual root cause.

AI-02 - Optimizer / trade-off narration

Explain active constraints, objective regressions, parameter movement, search-space boundaries, and robustness using Measurement and OptimizationRun data. The model never computes authoritative engineering numbers.

AI-03 - Template topology proposer + stronger retrieval

Retrieve by spec similarity, topology compatibility, validated outcome, robustness, and later post-layout outcome. Retrieve failures as well as successes so the system remembers what not to repeat.

AI-04 - Conversational Spec-to-Constraint

Translate fuzzy natural language into an unconfirmed SpecDraft with constraints, objectives, assumptions, ambiguities, missing inputs, source hash, and SI-normalized values. No silent defaults for supply, load, PVT, temperature, stability or measurement definitions.

AI-05 - Multi-objective Pareto analyst

Compute and persist a deterministic ParetoSnapshot containing objective definitions, selected trials, dominance relations, PVT/MC filter, and reproducibility IDs. AI answers "what do I give up?" questions from that snapshot.

Suggested typed SpecDraft

```
{ requested_function, constraints[], soft_objectives[], weighted_objectives[], assumptions[], ambiguities[],  
missing_inputs[], source_text_hash, needs_human_confirmation: true }
```

The easiest high-value additions are the features that turn existing structured data into better decisions. They create user-visible value before the expensive autonomy work begins.

14.3 V2-V3 intelligence: speed, multi-PDK, physical design, autonomy

AI-06 - Surrogate sizing accelerator

Start with the simplest model that wins on held-out data. Persist model/training hash, uncertainty, applicability domain, calibration, and residuals. High uncertainty or OOD automatically routes to real SPICE. Every promoted candidate is verified.

AI-07 - PDK porting assistant

Extract source design intent and operating points, map legal device classes/voltage domains, propose a target starting point, then run target validation -> SPICE -> optimization -> PVT/MC. Geometry ratios alone are not a port.

AI-08 - Multimodal DRC/LVS debugger

Consume structured DRC markers and LVS mismatch graphs first, then geometry/net metadata and images. Generate evidence-linked ECOProposal objects; applying an ECO creates a candidate revision and forces DRC/LVS again.

AI-09 - AI-assisted placement clustering

Generate LayoutConstraint proposals for symmetry, common centroid, matching, proximity, keep-apart, orientation, sensitive/high-swing nets, and verified PDK-backed well/guard constraints. Deterministic layout code places and routes.

AI-10 - Autonomous topology exploration

Model outputs CandidateCircuitIR -> graph/schema/unit/model checks -> sandboxed compile/simulation -> measurements/constraints -> novelty/dedup -> experimental log -> human review -> trusted-template promotion only after approval and benchmarks.

Sandbox floor for AI-10

- CPU and memory limits
- wall-clock timeout and cancellation
- bounded simulation count/output
- no unnecessary network
- provenance for every attempt
- experimental candidates cannot silently become trusted templates

14.4 Additional AI features beyond the original ten

ID	Feature	Why it matters
AI-11	Spec completeness / contradiction auditor	Find missing test conditions, conflicting hard constraints, unsupported metrics, and unit mistakes before optimization burns time.
AI-12	Active-learning experiment planner	Choose the next most informative simulations within a fixed budget; especially valuable once surrogate uncertainty exists.
AI-13	Sensitivity + bottleneck explainer	Narrate real perturbation/sensitivity runs so users see what device parameters actually move each metric.
AI-14	Yield / corner risk scout	Prioritize expensive PVT/MC regions for real simulation; predicted yield is never signoff yield.
AI-15	Regression + design-revision reviewer	Explain what changed across revisions, which specs flipped state, and what correlated with the regression.
AI-16	Testbench / measurement coverage assistant	Map every confirmed spec to a proving analysis/measurement and propose missing tests through normal validation.
AI-17	PEX / parasitic root-cause + ECO assistant	Join pre/post-layout metrics with extracted parasitics, rank candidate contributors, and propose re-verified ECOS.
AI-18	Knowledge-base curator / failure miner	Turn repeated validated history into candidate heuristics with support and counterexamples - never automatic design rules.
AI-19	Read-only natural-language ledger query	Ask engineering questions over validated history through an allowlisted query plan; no model-generated SQL execution.
AI-20	AI trust + regression benchmark harness	Sealed evaluations prevent prompt/model regressions and measure whether AI actually saves engineer time.

Product principle

The least flashy features - spec auditing, regression review, coverage, sensitivity, query, and evaluation - are likely to earn user trust earliest. The most flashy features - autonomous topology and visual layout AI - should arrive only after the engine, provenance and benchmark harness are mature.

14.5 Cross-cutting AI trust contract

- **No authoritative number without provenance.** A factual performance number points to a Measurement/SimulationRun/DRC/LVS/PEX artifact. Predictions are visibly labeled.
- **Structured before prose.** Design-changing intent is a typed object. Prose never becomes the command parser.
- **Explicit uncertainty and applicability.** Predictive features expose confidence/uncertainty and in-domain/OOD status.
- **Addressable evidence.** RAG citations are stable Experiment/Artifact/PDK-document IDs.
- **No hidden assumptions.** Spec generation and PDK porting surface assumptions for confirmation.
- **No AI signoff.** AI explains verification; it cannot replace SPICE/DRC/LVS/PEX/constraints.
- **No silent promotion.** Experimental candidates may auto-log, never auto-promote to trusted templates or committed revisions.
- **Fail closed on malformed model output.** Invalid schema is rejected or explicitly re-requested; never silently repaired then executed.
- **Record cost and latency.** AIAction includes enough metadata to measure workflow value against inference cost.
- **Model-agnostic.** LLMPProvider or a dedicated model interface is used everywhere; CI uses mocks.

The key boundary

AI proposes / predicts / narrates / retrieves / prioritizes -> Design Engine validates -> real EDA tools evaluate -> constraints judge -> human promotes trusted change

Why this matters for the startup

A model vendor can copy prompts. It is much harder to copy a growing corpus of reproducible circuit attempts, measured failure modes, accepted/rejected proposals, post-layout deltas, porting history, and calibrated model performance. The trust contract makes that corpus useful rather than noisy.

14.6 Additive data model and API expansion

Deferred entity	Owner	Purpose
SpecDraft	AI-04	Unconfirmed NL-to-Spec proposal with assumptions/ambiguities
ParetoSnapshot	AI-05	Reproducible non-dominated set tied to trial IDs
SurrogateModelRecord / Prediction	AI-06	Training domain, calibration, uncertainty and provenance
LayoutConstraint	AI-09	Matching/symmetry/proximity/keepout intent for deterministic layout
CandidateCircuitIR	AI-10	Untrusted novel topology candidate before compilation
AIExperimentPlan	AI-12	Proposed simulation batch, budget and selection score
LearnedHeuristic	AI-18	Evidence-backed candidate design rule with promotion state
AIEvaluationRun	AI-20	Sealed AI benchmark result by model/prompt/version

Additive Design Engine methods

```
propose_spec_draft() compute_pareto() plan_experiments() run_sensitivity()
predict_surrogate() port_design() propose_layout_constraints() diagnose_drc_lvs()
compare_post_layout() explore_candidate_topology() query_experiments() evaluate_ai_feature()
```

Do not create these tables in Stage 0. Each arrives only with its owning feature. Existing methods remain intact; new methods are additive and any design-changing method returns a proposal/revision candidate rather than mutating the committed design.

14.7 AI acceptance gates and measurable product value

V0.3 - Copilot

- zero unsupported factual claims on the sealed grounding suite
- ungrounded cases refuse rather than improvise
- SpecDraft exposes assumptions/ambiguities and requires confirmation
- Pareto narration matches deterministic snapshots
- model/prompt regressions are recorded in AIEvaluationRun

V0.4 - Topology intelligence

- every proposal cites valid experiment IDs
- retrieval includes relevant failures as well as successes
- proposal is schema-valid before simulation
- human accept/edit/reject decision is logged
- read-only ledger query cannot mutate state

V2.0 - Intelligence acceleration

- surrogate calibration + OOD thresholds pass held-out benchmarks
- target-PDK ports are re-optimized and verified in target PDK
- layout AI consumes structured verification evidence
- every AI ECO reruns deterministic physical verification

V3.0 - Autonomy

- restricted candidate IR and normal validators
- isolated execution with hard resource limits
- experimental candidates never auto-promote
- autonomy can be disabled without breaking the core product

Product-value checkpoint

For every feature, compare AI-assisted vs non-AI workflow on a fixed task: engineer time, number of EDA jobs, invalid proposals, factual/citation errors, human edits, inference cost/latency, and final verified design quality. Do not call the feature a win unless it measurably improves workflow without degrading correctness or trust.

14.8 Startup priority: what to build first

Priority	Features	Reason
A - earliest	AI-01, 02, 04, 05, 11, 13, 15, 16, 19, 20	Immediate trust and workflow value; uses data already needed for V1.
B - moat builders	AI-03, 12, 18	Turns experiment history into compounding retrieval/planning knowledge.
C - accelerators	AI-06, 07, 14, 17	High leverage only after enough validated data and multiple PDK/PEX flows exist.
D - post-V1 autonomy	AI-08, 09, 10	Impressive but expensive/risky; requires mature physical flow, provenance, sandboxes and benchmarks.

The strategic moat

Validated circuit engine + structured experiment ledger + failure history + topology templates + human decisions + post-layout outcomes + calibrated AI. The AI layer should continuously make that substrate easier to use and more valuable, not turn the product into one-shot LLM generation.

Keep the original sequencing

Do not reopen the Stage 0-4 architecture because the AI roadmap is exciting. The fastest route to these features is still the boring one: get the inverter, simulator, measurements, optimizer and provenance correct first. Then the AI has real evidence to work with.

One rule above the addendum

The agent proposes, the simulator decides, and you commit. Predictions accelerate engineering; they never replace verification.

15. Implementation Readiness Patch

September 2026 - final pre-implementation corrections. The architecture and AI expansion remain intact. These pages close the load-bearing details found by the final cross-document audit. Where older shorthand conflicts with pages 27-34 on these topics, this patch controls.

The seven mandatory corrections

- **Metric Contract + analysis/testbench matrix.** Every named measurement gets one formal meaning and one legal measurement context before code is trusted.
- **libngspice concurrency/isolation proof.** Stage 2 stress-tests 1, 2, and 4 simultaneous jobs; process isolation is the fallback without changing the Job API.
- **Statistical robustness protocol.** Monte Carlo claims carry N, seeds, corners, temperature/supply sets, actual PDK variation mechanisms, sampling method, thresholds, and uncertainty.
- **Reproducibility concepts separated.** Design identity, execution environment, reproducibility ID, and comparison tolerance are not collapsed into one ambiguous hash.
- **CACE is an adapter, not a second source of truth.** The Design Engine owns Specification, MetricContract, Measurement and verdict semantics.
- **AI data residency is explicit.** DISABLED / LOCAL_ONLY / HOSTED_ALLOWED; design-bearing hosted calls require opt-in and secrets are filtered.
- **Provider model IDs are configuration.** Operational prompts route through strong/fast capability aliases rather than stale hard-coded Gemini generations.

Two product corrections are now first-class too

- **UX is an acceptance surface.** A designer should complete the main flow without understanding Job, SQLite, LLMPProvider, or DesignEngine internals.
- **Physical backend ownership is proven, not assumed.** A one-device technology spike decides the practical KLayout/Magic/Netgen split behind adapters.

No fifth architecture review

After these contracts are present in the build plan and prompt playbook, the next useful artifact is still the real inverter running through the pinned environment. Implementation evidence now outranks more architecture debate.

15.1 Metric Contract + Analysis/Testbench Matrix

The Stage 3 weakness is semantic, not mathematical: gain, phase margin, slew rate, power, offset, and settling time are not interchangeable functions over arbitrary waveforms. Each measurement must be defined before it is automated.

Metric	Required analysis	Canonical testbench intent	Contract must freeze
DC / small-signal gain	DC or AC, explicitly selected	Amplifier or differential amplifier	gain type, reference, sign, nodes, units
Bandwidth	AC	Amplifier with declared reference gain	crossing rule, interpolation, search interval
Phase margin	Loop gain / return ratio	Defined closed-loop feedback benchmark	loop break/injection method, crossover definition
Slew rate	Large-signal transient	Declared step-response benchmark	stimulus, measurement region, polarity
Power	Operating point and/or transient average	Circuit-specific supply measurement	current sign, supply set, averaging/integration window
Offset	DC; mismatch/MC only when claimed	Differential-input benchmark	offset methodology, sweep/solve rule, variation context
Settling time	Transient	Closed-loop step	final value estimator, error band, staying condition

MetricContract minimum schema

metric_id/version; semantic definition; Analysis type; testbench capability; stimulus/operating condition; observed ports/nodes; extraction algorithm; sign convention; SI units; measurement window/reference; interpolation/crossing rule; invalid-data behavior; comparison policy; golden-fixtue ID.

Human checkpoint

The first implementation of each MetricContract is verified against its appropriate benchmark and an independent hand calculation or equivalent reference. A current mirror is not accepted as a phase-margin benchmark merely because it is already in the test suite.

15.2 Simulation Isolation + Reproducibility

The Job abstraction stays single-machine and small. What changes is the assumption that the libngspice shared library is automatically safe under concurrent in-process use. Stage 2 must prove the selected wrapper/runtime behavior.

Mandatory Stage 2 isolation suite

- Repeated 1-job, 2-job, and 4-job simultaneous workloads.
- Mix different circuits, seeds, callback streams, cancellation timings, and output sizes.
- Assert that results/waveforms never appear under the wrong Job ID.
- Assert no parameter, callback, simulator-state, or cleanup leakage between runs.
- Assert cancellation/failure of one Job does not corrupt another.
- If any test fails, keep the public Job/Simulator contract and move concurrent execution to isolated worker processes. Do not weaken the test.

Canonical identity model

```
design_identity_hash = SHA256(canonical_encode({normalized_netlist, logical_simulation_config}))
execution_environment_hash = SHA256(canonical_encode({simulator_build, pdk_version, model_hashes, container_digest,
backend_versions, measurement_version}))
reproducibility_id = SHA256(canonical_encode({design_identity_hash, execution_environment_hash, random_seed,
analysis_settings}))
comparison_policy_id = versioned numerical-equivalence / tolerance policy
```

Important: numerical tolerance is a comparison policy, not a property of circuit identity. Floating-point simulation output is compared numerically under that policy; it is never required to be byte-identical.

15.3 Statistical Robustness Protocol

The Stage 4.5 target is not a universal “95% pass” number. The target is a reproducible, auditable statistical claim under an explicitly declared protocol.

Every robustness report carries

- N and pass/fail counts;
- random seed(s) and sampling method;
- process corner set, supply set, and temperature set;
- which variation mechanisms actually came from the pinned PDK/model cards, distinguishing process variation and mismatch when applicable;
- metric threshold(s) and the MetricContract versions used;
- yield estimate plus confidence interval or explicitly justified uncertainty method;
- PDK/model hashes, environment/reproducibility IDs, and the exact Experiment IDs behind the aggregate.

Example of an acceptable headline

“Under StatisticalProtocol MC-07: N=1000, seed set S-12, declared corners/supply/temperature, and the pinned model variation mechanisms, 972/1000 samples met gain ≥ 55 dB; estimated yield 97.2%, with the declared confidence interval. See EXP-... and protocol artifact.”

The exact percentage is an observed result, not the stage-completion criterion. Any “X% pass” statement still triggers the existing human robustness checkpoint before it is promoted as a headline result.

15.4 The Real Product: First-Time Designer Journey

The engine is the foundation, but a real user should not need to know its internal nouns. V0.4/V1.0 now has an explicit UX acceptance flow.

Primary journey

New Project -> choose PDK / supply / AI privacy mode -> describe circuit intent or choose a template -> enter specs -> detect missing assumptions / contradictions / unmeasurable constraints -> confirm Specification -> propose one or more known topologies with evidence -> inspect schematic -> size/optimize -> compare Pareto candidates -> inspect failures and sensitivity -> accept/branch a DesignRevision -> layout -> DRC/LVS/PEX -> compare pre- vs post-layout measurements -> export reproducible artifacts/report

Minimum views

View	User question it answers
Project Home	What am I designing, under which PDK, and what is currently verified?
Spec Builder	What exactly must pass, what assumptions are missing, and can each spec be measured?
Design Workspace	What topology/revision am I looking at and what changed?
Run Center	What is running, what failed, and what measurements/waveforms came back?
Explore / Optimize	Which candidates are best and what trade-offs/sensitivities exist?
Evidence Inspector	What real experiment or artifact supports this AI statement/proposal?
Physical Verification	Where are DRC/LVS/PEX issues and what changed after layout?
Decision Gate	Accept, reject, compare, or branch - never silently commit.

UX done-when

A new user can complete the benchmark two-stage amplifier journey on a clean install without touching the database, Job model, LLMProvider, or Design Engine API, while always being able to answer: what did the tool try, why, what measured truth supports it, what failed, and what should I do next?

15.5 Physical Backend Proof + CACE Boundary

KLayout remains the preferred viewer/geometry candidate, but the architecture should not confuse “primary” with “must perform every physical operation.” The existing adapter boundary is specifically what allows evidence-driven tool choice.

Stage 8 technology spike

one device -> programmatic geometry -> DRC -> extract -> LVS -> PEX -> pre/post-layout simulation comparison

- Run every step through LayoutBackend / defined verification-extraction adapters.
- Record exact tool/deck versions and artifacts for each step.
- If KLayout is excellent for viewing/geometry but Magic/Netgen is more reliable for an operation, use the better adapter. Do not special-case around the Design Engine.
- Only after the spike should Stage 8 freeze the practical responsibility split.

CACE: useful integration. one canonical authority

```
Design Engine canonical Specification + MetricContract + Measurement  
-> CACE adapter / export / execution bridge  
-> import raw/results  
-> normalize into canonical Measurement / Experiment rows
```

CACE does not become a parallel definition of gain, limits, units, or pass/fail. Round-trip tests must detect semantic loss between the canonical MetricContract and the adapter representation.

15.6 AI Data Residency + Provider Model Aliases

A proprietary circuit can contain valuable schematic structure, sizing, failures, experiment history, and layout information. Hosted AI therefore becomes an explicit project-level decision rather than an invisible default.

Mode	Behavior
DISABLED	No local or hosted model calls. Core EDA/design workflow remains usable.
LOCAL_ONLY	Default for design-bearing contexts. Only approved local providers/models; no design payload leaves the machine.
HOSTED_ALLOWED	Hosted calls are allowed for this project/workspace only after explicit user opt-in and provider disclosure.

- Before the first hosted call, show provider/model and what classes of design artifacts will be sent.
- Switching to LOCAL_ONLY immediately blocks the next hosted call - no restart required.
- API keys, environment variables, PDK credentials, license tokens, and configured secrets are filtered from prompt payloads and never stored in AIAction provenance.
- Every hosted AIAction still records provider/model/prompt version and source-artifact classes for auditability.

Model maintenance rule

```
GEMINI_STRONG_MODEL =  
GEMINI_FAST_MODEL =
```

Operational prompts route by capability alias. Exact provider model IDs live in config and must pass availability + sealed-benchmark checks before promotion. Provider churn becomes a config/benchmark event instead of a documentation rewrite.

15.7 Claim Discipline and the Final Next Action

The technical ambition does not shrink, but V1 language must describe what the open PDK/toolchain actually proves.

Recommended V1 wording

- **Use:** “spec-to-physical-verification” or “spec-to-verified-design” for the end-to-end Sky130 flow.
- **Avoid:** unqualified “production signoff” claims until the complete flow has evidence supporting that term.
- **Avoid:** claiming a general professional Virtuoso/Spectre replacement in V1. Position it as an open, reproducible, AI-native environment for a defined class of open-silicon analog designs.
- **Stage 9 physical verification** means the platform completed its declared DRC/LVS/PEX/post-layout checks under the pinned open PDK/toolchain; it is not a foundry certification statement.

Final frozen sequence

Apply the v3.2/v1.2 contracts -> Stage 0 packaging -> Stage 1 circuit kernel -> Stage 2 real inverter + isolation/reproducibility proof -> Stage 3 MetricContracts -> Stage 4 optimization -> Stage 4.5 declared robustness -> AI/GUI/physical flow in the already defined order.

Do not perform another giant architecture pass. The next decisive question is empirical: can the real pinned environment carry the inverter through schema -> validation -> netlist -> libngspice -> waveform -> reproducibility/isolation tests, and then can a real designer prefer the resulting workflow over the fragmented open tools they use today?

Architecture status after this patch: FROZEN FOR IMPLEMENTATION.